

비전+음성 온디바이스 감정·인지 상태 실시간 모니터링 연구 설계서

요약

본 연구는 Raspberry Pi M.2 HAT+ + Hailo-8L(13 TOPS) 기반 온디바이스 시스템에서 **감정(Valence/Arousal) + 인지부하(mental demand/effort 등)**를 실시간으로 추정하는 “논문 제출·포트폴리오 데모·벤치마크”까지 바로 이어지는 확정 설계를 제시한다. 핵심 전략은 (i) 정답 라벨이 명확한 공개 AV 데이터셋 **AVCAffe(108h, 58k+ clips, 106명)**으로 정량 평가를 보장하고, (ii) Hailo-8L은 **CNN 추론(비전/스펙트로그램)**에 집중시키며 STFT·VAD·(선택)ASR 등 일부 전처리는 CPU에서 수행, (iii) ONNX→Hailo DFC(HAR)→Optimize(INT8)→Compile(HEF)→HailoRT/TAPPAS(GStreamer)로 배포·실측하며, (iv) 정확도(Acc/F1/AUC/MAE)와 실시간성(FPS/latency/p50·p95/power/memory)을 결합 보고한다. ¹

연구 목표, 가설, 기여

목적(평가 가능성 기준 확정)

카메라·마이크 입력으로부터 2초 단위로 (a) **감정(valence, arousal)** 과 (b) **인지부하(mental demand, effort, temporal demand 중 최소 2개)**를 동시에 추정하고, Hailo-8L 가속으로 **실시간 지연·전력 예산을 만족**하는 파이프라인을 만든다(학습·평가는 AVCAffe, 실시간성 평가는 Pi 실측). ²

가설

- H1: 동일한 subject-independent split에서 **멀티모달(비전+오디오) late fusion**이 단일모달 대비 AUC/F1(분류) 또는 MAE(회귀)를 유의미하게 개선한다(AVCAffe가 affect+CL 동시 라벨을 제공). ³
- H2: Hailo DFC 기반 INT8(PTQ→필요 시 QAT) + 지식증류를 적용하면 FP32 대비 성능 저하를 제한하면서(예: $\Delta AUC \leq 0.02$ 또는 $\Delta MAE \leq 5\%$) 지연/FPS/전력을 달성한다(Profiler 예측 + Pi 실측 비교). ⁴
- H3: **VAD 기반 오디오 게이팅 + 프레임 드롭/버퍼 정책**을 적용하면 p95 지연이 안정화되고(큐 적체 감소), 장시간 실행에서 FPS 변동이 줄어든다(TAPPAS의 queue/flush 개념을 운영정책에 반영). ⁵

기여(논문화 포인트)

- “정서+인지부하”를 동일 스트림에서 다루는 **멀티태스크·멀티모달 설계**(AVCAffe baseline을 재현 기준선으로 삼고, 온디바이스 제약 하 설계 변경을 정량화). ⁶
- Hailo-8L에서 **멀티 네트워크(오디오CNN+비전CNN)**를 운영하는 실시간 엔드투엔드 구현(파이프라인 동기화·버퍼링·장애 복구 포함)과 벤치마크 프로토콜. ⁷
- DFC Profiler의 추정치(FPS/전력 등)와 Pi 실측치 간 괴리를 **Bland-Altman/상관 분석**으로 보고하여 “예측 기반 설계”의 신뢰구간을 제시. ⁸

데이터 설계

공개 데이터셋 확정 목록과 역할

우선 순위	데이터셋	모달리티	라벨(정답)	규모/특징	연구 내 역할	리스크/대응
1	AVCAffe	오디오 + 비디오 (+face crops)	valence/arousal + cognitive load(mental demand/temporal demand/effort/...; NASA-TLX·SAM 기반)	108h, 58k+ 6초 clips, 106명; 짧은 세그먼트 · face-crops · split 파일 제공	메인 학습·평가(정량)	접근 승인 필요 · 기간 제한 가능 → 초기에 승인/다운로드 자동화·버전 고정 ⁹
2	CREMA-D	오디오 + 비디오 (및 평가 라벨)	6 감정(행복/슬픔/분노/공포/혐오/중립)	7,442 clips, 91명(20~74세)	오디오/비전 인코더 사전학습 · 외부 타당성	acted 데이터 편향 → fine-tune는 AVCAffe로 제한 ¹⁰
3	RAVDESS	오디오-비디오/오디오-only/비디오-only	감정(강도 포함)	7,356 recordings, 24명, 3 모달 조건 제공	단일모달 vs 멀티모달 구조 검증	배우 기반 → “일반화 한 계”로 명시 ¹¹
4(선택)	AI-Hub 감정 대화 음성	오디오 (+메타)	7감정, 메타에 음성 인식 결과 포함	48kHz wav, csv 메타(상황 · ASR결과·감정·성별/나이)	(선택) 한국어 음성 감정·ASR 품질(WER/CER) 평가	신청/정책 제약 가능 → “선택 트랙”으로 설계 ¹²
대체	ADABase	멀티모달 (행동/생리/설문 등)	인지부하 레벨/연속 값(연구에서 공개 릴리스)	30명 공개 릴리스 언급	AVCAffe 접근 실패 시 인지부하 대체 벤치마크	오디오 포함 여부 불확실 → 비전+생리로 축소하거나 오디오 별도 데이터 결합 ¹³

데이터 “수집 대체 계획”과 접근·윤리

- AVCAffe는 **Dataverse 계정·API 토큰 기반 접근 절차**(리포에 단계별 안내·다운로드 제공)가 있으므로, 연구 리스크는 “승인 지연/기간 제한”이다. 대응: (i) 승인 즉시 전체 다운로드 자동화, (ii) 원본은 read-only 보관, (iii) 파생 산출물(스펙트로그램, 얼굴 crop index, split manifest)만 재배포 가능 형태로 관리한다. ¹⁴
- AVCAffe는 `public_face_ids` (얼굴/이미지 공개 동의 참가자 목록)와 같은 메타를 제공한다. 데모/논문 그림 사용 시 **해당 ID만 사용**하거나, 원칙적으로 **얼굴 모자이크/익명화**를 기본으로 한다. ¹⁵
- 본 연구는 “공개 데이터 기반 모델”이라도, 포트폴리오 데모에서 **실사용자 얼굴·음성**을 받는 순간 개인정보 처리 이슈가 발생한다. 데모 모드에서는 (i) 기본값 “저장 안 함”, (ii) 저장 시 명시적 동의, (iii) 로컬 암호화 저장 (예: LUKS/파일단 AES) 후 즉시 삭제 옵션을 제공한다(윤리 섹션에 동의서 예시 포함). ¹⁶

포맷·라벨링 지침(학습/평가용 확정 스키마)

학습 샘플 단위(고정): “2초 윈도우(오디오 16kHz, 비디오 16fps)”를 기본 샘플로 하되, AVCAffe의 6초 clip은 2초×3개로 슬라이싱(겹침 0.5초 옵션)한다. 이때 **라벨은 원 clip 라벨을 상속**하고, 슬라이싱으로 인한 label noise는 **Huber(회귀) + label smoothing(분류)**로 완화한다. ¹⁷

라벨 정의(최소 확정 출력)

- 회귀 출력: valence, arousal, mental_demand, effort (4개)
- 분류 출력(평가 지표 충족): 각 회귀 라벨을 **훈련셋 분위수(q33/q66) 기준 3클래스(low/med/high)**로 변환한 파생 라벨을 추가 생성(Acc/F1/AUC 보고용). (분류는 “정답이 바뀌는” 것이 아니라 “회귀 라벨의 평가 관점”을 분리한 것이므로 재현 가능). ⁶

전처리·증강(확정 파라미터)

- 비디오: face crop → 224×224, 16fps, 2초(32프레임) 입력. AVCAffe baseline도 16fps 다운샘플 및 face crop 사용을 명시한다. ¹⁷
- 오디오: 16kHz로 리샘플, 2초 파형 → log-mel(예: 80 mel, hop 10ms 기반) + (선택)SpecAugment(time/freq mask). SpecAugment는 스펙트럼 마스킹 기반 증강으로 널리 사용된다. ¹⁸
- 증강(비디오): random horizontal flip, color jitter 등은 AVCAffe baseline에 포함. ¹⁹
- 증강(오디오): volume jitter(AVCAffe baseline) + (선택)SNR-based noise mix(실사용 환경 근사). ¹⁹

저장·보안(재현 가능한 운영 규정)

- 저장 폴더 규칙: raw/ (원본), derived/ (멜/프레임 인덱스), splits/ (참가자 단위 split manifest), models/ (FP32/INT8/HEF), logs/ (벤치·학습 로그).
- 해시 기반 무결성: 각 산출물에 sha256을 기록하고, HEF에도 hailortcli 기반 메타 덤프를 함께 저장(버전 드리프트 추적). DFC/HailoRT는 CLI 도구 제공을 전제한다. ²⁰

모델 및 학습 설계

입력·출력·융합 방식(온디바이스 제약에 맞춘 확정)

기본 전략: “듀얼 인코더 + CPU late fusion + 멀티태스크 헤드”

- 비전 인코더(HEF#1): 얼굴 crop 시퀀스를 “프레임 단위 CNN + temporal pooling”으로 처리
- 오디오 인코더(HEF#2): log-mel(1ch)을 2D CNN으로 처리
- 융합/헤드(CPU): concat(비전 임베딩, 오디오 임베딩) → MLP → 4개 회귀 + 4개 3클래스 분류(파생 라벨)

이 분해는 Hailo NPU가 “신경망 추론”에 최적화되어 있고(Raspberry Pi 문서), 다중 네트워크를 GStreamer 파이프라인에서 병렬 추론할 수 있다는 전제(“hailonet multiple times”)와 맞물린다. ²¹

아키텍처(권장 백본과 Hailo 호환성 근거)

- 비전 백본: MobileNetV3-Small(또는 EfficientNet-lite0) + temporal pooling(평균/attention-lite)
- 오디오 백본: MobileNetV3-Small(입력 채널=1) 또는 “첫 conv만 1ch로 수정한 EfficientNet-lite”
- 활성화함수/레이어 선택 근거: Hailo DFC는 ReLU/SiLU/Swish/Mish/Hard-swish(프리뷰) 등 다수 activation을 지원하며, 일부는 “preview”로 명시된다. 따라서 **Hard-swish 의존이 큰 구성(MobileNetV3)**은 컴파일 단계에서 검증이 필요하고, 리스크 회피가 필요하다면 ReLU/SiLU 기반 백본으로 대체한다. ²²

시간 모델링은 LSTM/RNN을 피하고(피드백 루프 불가→unrolling 필요, 길이 증가 시 성능 저하 가능) **짧은 시퀀스(≤32프레임) pooling**으로 고정한다. ²³

손실함수(멀티태스크)

- 회귀: Huber(기본) + MAE 보고
- 분류: Cross-Entropy(+ class weight; q33/q66로 class imbalance 조절)
- 멀티태스크 가중치: $L = \sum_i \lambda_i L_{reg_i} + \sum_j \mu_j L_{cls_j}$ (λ/μ 는 검증셋에서 고정; 논문에 명시)

학습 스케줄·데이터 분할·교차검증(평가 가능성 최우선)

- 분할 원칙: **participant-independent** (동일 참가자 클립이 train/test에 섞이지 않도록). AVCAffe는 train/val 참가자 목록 파일을 제공한다. 15
- 최종 스킴(확정):
- Fold-0: 제공 train/val 기반 + 추가 test split(참가자 단위)
- Fold-1..4: 참가자 단위 5-fold CV(옵션이 아니라 “논문용 기본”).
- 학습 설정(재현 기준선): AVCAffe baseline은 Adam + warm-up multi-step LR, 16fps, 2초 입력, mel-spectrogram 변환 등을 제시한다. 본 연구는 이를 **재현 베이스라인**으로 삼고, 온디바이스용 경량 백본으로 교체하는 ablation을 수행한다. 19

베이스라인 모델(논문 비교군 확정)

- 데이터셋 제공 baseline(재현): AVCAffe appendix의 백본/전처리/스케줄을 최대한 재현(서버 FP32). 6
- 단일모달: 비전-only, 오디오-only (동일 백본 파라미터 수 맞춤)
- 고전 ML:
- 오디오: MFCC 통계 + SVM/LogReg
- 비전: 얼굴 랜드마크 통계(선택) + XGBoost
- 배포 비교군: Pi CPU-only(ONNXRuntime) vs Hailo INT8(HEF) vs (가능 시) 다른 NPU(예: EdgeTPU)

경량화 및 Hailo-8L 최적화

제약·가정(명시)

- Hailo-8L 기반 AI Kit는 13 TOPS NPU 모듈(M.2 2242)이며, 카메라 스택과 통합되는 형태로 제공된다. 24
- DFC는 ONNX/TF 모델을 Hailo 내부 표현(HAR)로 변환하고, **activation statistics 기반으로 FP32→INT8 최적화 후, HEF(바이너리)**를 생성한다. 25
- STFT/log-mel, VAD, 오디오 I/O, (선택)ASR은 CPU에서 수행하고, Hailo-8L에는 “스펙트로그램 CNN/비전 CNN”만 올린다(연산 지원·운영 리스크 최소화). 26
- 모델 컴파일(DFC/HEF 생성)은 보통 GPU/RAM 요구가 크므로 x86 리눅스에서 수행하는 것을 기본으로 한다 (공식 흐름도에서 optimize에 GPU 권장). 27

지식증류·프루닝·설계 변경(실행 절차 확정)

- 지식증류(Teacher→Student): Teacher는 ResNet18/ConvNeXt-Tiny 등 서버 FP32, Student는 MobileNetV3 계열로 두고, soft target을 이용해 Student 성능을 유지하는 방식은 지식증류 고전 기법으로 정립되어 있다. 28
- 프루닝: 구조적 채널 프루닝(Conv 채널 감소) → 미세조정 → 양자화 단계로 넘기는 순서를 권장(“pruning + quantization” 조합은 대표 압축 파이프라인으로 제시). 29
- 네트워크 설계 변경(온디바이스 친화):
- temporal modeling은 pooling으로 단순화(LSTM unrolling 리스크 회피) 23
- BN folding, ReLU/SiLU 중심으로 단순화(양자화 안정성)
- 마지막 레이어에서 불필요한 Reshape/후처리를 제거하고 end_node를 “마지막 신경망 연산 직전”으로 잡는 것이 컴파일 이슈 회피에 중요하다(특히 postprocess 구간 분리). 30

PTQ/QAT 및 DFC 최적화 파이프라인(ONNX→DFC→HEF 확정)

DFC CLI 기준(재현용 명령 흐름)

- Parse(ONNX→HAR): `hailo parser onnx ...` 는 ONNX/TF를 HAR로 변환하는 CLI 도구로 명시. ³¹
- Optimize(INT8): `hailo optimize` 는 최적화/양자화를 수행하며, calibration dataset은 “recommended > 1024”로 제시된다. ³²
- Compile(HEF): `hailo compiler` 로 HAR→HEF. ³¹

중요 디테일(정확도·실패율을 좌우)

- DFC는 calibration을 위해 native emulation으로 activation statistics를 수집해 8-bit 표현을 만든다고 설명한다. 따라서 calibration set은 실제 입력 분포(얼굴 crop/멜)에서 샘플링해야 한다. ³³
- HailoRT는 모델을 로드/실행하는 런타임이며, PCIe 연결 시 PCIe 드라이버로 통신한다. C/C++ 및 Python API(pyHailoRT)와 CLI(hailortcli)를 제공한다. ³⁴

TAPPAS/GStreamer 통합(멀티 네트워크 운영 설계)

- TAPPAS는 GStreamer 기반 파이프라인 예제(예: `gst-launch-1.0 ... hailonet hef-path=...`)를 제공하며, “multinetworks_parallel”처럼 단일 스트림에서 두 **hailonet** 으로 다중 네트워크를 돌리는 구조를 문서화한다. ³⁵
- Hailo 공식 소프트웨어 페이지는 GStreamer 플러그인(hailonet)이 “파이프라인에서 여러 번 사용되어 다중 네트워크 병렬 추론”이 가능하다고 명시한다. ³⁶

연산·메모리·지연·전력 예측 및 실측(방법 확정)

- DFC Profiler는 HAR를 기반으로 HW 자원·FPS 등을 프로파일링하며(레이어별 breakdown 포함), 보고서에 **NN Core 전력 추정(단일 context 소형 모델 한정, ±20% 정확도, 25°C 기준, 인터페이스 전력 제외)** 필드를 제공한다. 예측치는 “설계 단계 의사결정”에 사용하고, 논문에서는 실측으로 보정한다. ³⁷
- 예측↔실측 비교: 모델별(최소 3개 변형: FP32, PTQ INT8, QAT INT8)로 (Profiler 예측, Pi 실측)을 쌍으로 저장하고, 방법 비교는 Bland-Altman으로 시각화한다. ³⁸

전처리 및 실시간 파이프라인 설계

동기화·버퍼링(온라인 추론 규칙)

- 기준 시간: monotonic clock를 기준으로 오디오 프레임·비디오 프레임에 타임스탬프 부여
- 버퍼 구조:
 - 오디오: 16kHz PCM ring buffer(예: 10초)
 - 비디오: 프레임 큐(예: 최대 1초; 초과 시 drop)
- 윈도우 정렬: 매 200ms마다 “현재 시각 t”에 대해 [t-2.0s, t] 구간 오디오를 잘라 멜 생성, 비디오는 마지막 2초(32프레임)를 sampling (프레임이 부족하면 zero-pad/이전 프레임 반복)
- 큐 적체 대응: “p95 지연”을 망치는 병목은 queue 누적이므로, TAPPAS 예제에서 보이는 queue/leaky 개념을 정책으로 채택(프레임 드롭을 허용하되 지연 폭주를 방지). ³⁹

VAD(음성 구간 검출)와 오디오 게이팅

- WebRTC VAD는 voiced/unvoiced 분류를 수행하는 경량 VAD로 널리 사용되며, 실시간 파이프라인에서 무음 구간의 불필요한 추론을 줄이는 데 적합하다. ⁴⁰
- 운영 규칙(확정): 2초 윈도우에서 voiced ratio < 0.2이면 오디오 인코더 추론을 skip하고 “오디오 임베딩=0 벡터 + 품질 플래그”로 전달(멀티모달 결합 시 오디오 가중을 낮추는 효과).

face align(실시간 얼굴 정렬)과 검출 실패 처리

- 학습(오프라인)은 AVCAffe face crop을 사용해 “감정/인지 모델의 핵심”을 먼저 안정화하고, 실시간 데모에서만 얼굴 검출/정렬을 추가한다(오프라인 평가의 변동성 최소화). 41
- 실시간 처리(확정):
- 얼굴 검출(별도 HEF, 예: 경량 face detector) → crop
- (선택)landmark 기반 정렬 → 감정/인지 비전 인코더 입력
- 검출 실패 시: 이전 프레임 crop 유지($\leq 0.5s$), 그 이상은 “no-face” 상태로 마스킹(출력에 불확실성 ↑ 표기)

ASR 전사 품질(WER/CER/RTF)과 특징 정합(선택 트랙)

본 프로젝트는 “비전+오디오”가 본체지만, **시니어 마인드케어 응용**에서 음성 내용(말더듬/반응지연/어휘 등)로 확장될 가능성이 높으므로, 포트폴리오 차별화를 위해 “선택 실험”으로 ASR 품질을 함께 리포팅한다.

- Whisper는 코드/가중치가 MIT License로 공개되어 로컬 추론이 가능하다. 42
- WER 정의는 NIST 평가 문서에서 “(삭제+삽입+치환)/참조단어수”로 제시된다. 43
- CER는 torchmetrics 문서에서 (S+D+I)/N 형태로 제시된다. 44
- RTF는 처리시간/오디오길이의 비로 정의된다(임베디드 음성 성능 평가에서 활용). 45
- 기준 데이터(선택): LibriSpeech(약 1000h, 16kHz)로 영어 ASR 속도/정확도 벤치마크를 고정한다. 46

구현 설계

소프트웨어 스택(확정)

- OS: Raspberry Pi 5 + 64-bit Raspberry Pi OS 기준(공식 AI Kit setup 가이드가 Bookworm 64-bit를 전제로 하고, 설치/검증 커맨드를 제시). 47
- 필수 설치: `sudo apt install hailo-all` 로 커널 드라이버/펌웨어, HailoRT, TAPPAS core, rpicam-apps Hailo postprocess를 설치하고, `hailortcli fw-control identify` 로 장치 인식을 확인한다. 48
- 성능 설정: PCIe Gen 3.0 활성화는 “옵션이지만 성능상 강력 권장”으로 명시된다. 49
- 버전 고정: Raspberry Pi 문서는 “Hailo toolchain/driver 버전 불일치 시 동작하지 않을 수 있음”을 경고한다(릴리스/포트폴리오에서 가장 흔한 장애 원인). 50

코드 구조(예시 리포지토리 스캐폴딩)

```
mindcare-av/  
  configs/  
    dataset_avcaffe.yaml  
    model_audio.yaml  
    model_video.yaml  
    hailo_compile_audio.yaml  
    hailo_compile_video.yaml  
  src/  
    data/  
      avcaffe_download.py  
      avcaffe_preprocess.py  
      splits.py  
    models/  
      audio_cnn.py  
      video_cnn.py
```

```

    fusion_head.py
train/
    train_fp32.py
    distill.py
    prune.py
    qat.py
deploy/
    export_onnx.py
    build_hef.sh
    pi_runtime/
        video_gst.py
        audio_rt.py
        fusion_runtime.py
        metrics_runtime.py
scripts/
    benchmark_pi.sh
    eval_offline.py
    make_report.py

```

HEF 로드(파이썬) 스니펫(구현 가이드)

HailoRT는 host에서 동작하는 런타임 라이브러리이며 pyHailoRT(Python API)를 제공한다. 또한 Hailo-8L을 쓰려면 hailort 리포에서 `hailo8` 브랜치를 사용하라는 안내가 있다. ⁵¹

```

# 개념 스니펫: 실제 클래스/메서드명은 설치된 pyHailoRT 버전에 맞춰 조정
# 목표: (1) HEF 로드 (2) 입력/출력 스트림 준비 (3) 비동기 추론 (4) timestamp 포함 반환

from pathlib import Path
import numpy as np
import time

HEF_PATH = Path("models/video.hef")

def infer_one_window(hailo_device, input_tensor_uint8: np.ndarray) -> np.ndarray:
    """
    input_tensor_uint8: (H,W,C) 또는 (1,H,W,C), uint8
    returns: embedding or logits, np.ndarray
    """
    # 1) network_group = hailo_device.load_hef(HEF_PATH)
    # 2) vstreams = network_group.create_vstreams(...)
    # 3) vstreams.input.send(input_tensor_uint8)
    # 4) out = vstreams.output.recv()
    # return out
    raise NotImplementedError

def now_ms():
    return int(time.time() * 1000)

```

GStreamer/TAPPAS 파이프라인(비전 스트림) 스니펫

TAPPAS User Guide는 `gst-launch-1.0 ... hailonet hef-path=...` 형태의 파이프라인과 multi-network 예제를 제공한다. ⁵²

단일 네트워크(비전 인코더) 예시

```
gst-launch-1.0 \
  libcamerasrc ! videoconvert ! videoscale ! \
  video/x-raw,width=224,height=224,pixel-aspect-ratio=1/1 ! \
  queue ! hailonet hef-path=video.hef qos=false ! \
  fakesink
```

시스템 아키텍처(확정 다이어그램)

```
[Pi Camera(libcamera/rpicam)] -> [GStreamer] -> [Face detect/crop(선택)] ->
[hailonet: video.hef] -> video_emb,t_v
[Mic(ALSA)] -> [RingBuffer] -> [VAD] -> [log-mel CPU] -> [pyHailoRT:
audio.hef] -> audio_emb,t_a
(video_emb, audio_emb, quality_flags, Δt=|t_v-t_a|) -> [CPU late-fusion head]
-> {valence, arousal, mental_demand, effort} + 위험지수
-> (UI: 웹 대시보드/로컬 GUI) + (CSV/Parquet 로그)
```

실험, 재현성, 윤리, 일정, 참고문헌

실험·평가 설계(지표·통계·비교군·ablation)

정확도 지표(오프라인)

- 회귀: MAE(필수)
- 분류(파생 3클래스): Acc, Macro-F1, ROC-AUC(OVR)
- 통계검정:
- AUC 비교: DeLong(상관 ROC) ⁵³
- 이진 분류 비교(파생 “high-risk” 이진화 실험 시): McNemar ⁵⁴

실시간·시스템 지표(온디바이스)

- latency: end-to-end p50/p95(윈도우 단위)
- FPS(비디오 파이프), RTF(선택 ASR/오디오 처리; 처리시간/오디오길이) ⁴⁵
- power: USB 전력계/INA 계열로 보드 입력 전력 측정 + DFC NN core 전력 추정과 비교(±20% 가정) ²³
- memory: RSS/heap(프로세스별), HEF 로드 후 peak

비교군(확정)

- 서버 FP32(AVCAffe baseline 재현) vs Pi CPU-only(ONNXRuntime) vs Pi+Hailo INT8(HEF)
- 다른 NPU는 “가능 시” 옵션으로 분리(보유하지 않아도 논문 완결 가능)

Ablation(최소 세트 고정)

- 모달리티: 비전-only / 오디오-only / fusion
- 압축: FP32 / PTQ INT8 / QAT INT8 / (QAT+KD)
- 전처리: VAD off/on, 프레임 드롭 정책 A/B, 얼굴 정렬 off/on

실험 계획표(표)

단계	산출물	핵심 설정(고정)	평가 포인트
데이터 접근·전처리	동일 manifest + split 파일	AVCAffe 참가자 단위 split, 2초 윈도우	재현 가능한 샘플링/라벨 상속 로직
FP32 기준선	AVCAffe baseline 재현 결과	16fps, 16kHz, mel 기반, Adam 스케줄	논문 “기준선” 확보 19
경량 학생모델	MobileNet 계열 + fusion	동일 split, 동일 헤드	정확도 대비 파라미터/OPS
압축+양자화	PTQ→QAT→KD	calib≥1024, hailo optimize/compile	성능저하 vs 지연/전력 55
온디바이스 통합	실시간 데모 앱	hailo-all, PCIe Gen3, hailonet/pyHailoRT	p95 지연, FPS, power 56
논문 패키징	재현 컨테이너+로그	seed 고정, commit hash, HEF 해시	재현성 체크리스트 통과

구현 체크리스트(단계별)

- 환경 고정: OS/패키지 버전 기록, PCIe Gen3 활성화, `hailo-all` 설치, `hailortcli fw-control identify` 성공 로그 저장 48
- 데이터: AVCAffe 접근 승인→다운로드 스크립트 실행→`info/train.txt` 기반 split manifest 생성→`no_audio_files` 처리 15
- 학습: baseline 재현(FP32)→학생모델 학습→KD/프루닝→QAT
- 배포: ONNX export→`hailo parser/optimize/compiler` 로 HEF 생성(타겟 `hw_arch=hailo81`)→Pi 복사 25
- 런타임: 비전(GStreamer hailonet) + 오디오(pyHailoRT) + CPU fusion → 타임스탬프 동기화/버퍼 정책 적용 57
- 벤치: latency(p50/p95)/FPS/power/memory 자동 로그 + 프로파일러 예측치 병렬 저장 23

예상 결과·해석 가이드(표·차트 제안)

관찰	해석	다음 실험	추천 차트
fusion이 MAE ↓ / AUC ↑	상보 정보 결합(H1 지지)	“오디오 게이팅” ablation	모델별 ROC + MAE 막대
PTQ에서 성능 급락	calib 분포 불일치/양자화 노이즈	QAT + KD로 회복	FP32 vs INT8 scatter(충별 noise) 58
p95 지연 폭주	큐 적체/동기화 실패	leaky queue/드롭 정책 조정	latency 히스토그램 + 타임라인 39
Profiler 전력 예측과 괴리	인터페이스/호스트 전력 포함 여부	보드 입력전력 vs NN core 분리 보고	Bland-Altman 8
(선택)ASR WER ↑, RTF>1	Pi에서 실시간 전사 한계	모델 tiny/basesmall 교체·부분 전사	WER/CER-RTF 트레이드오프 59

일정·자원(마일스톤·리스크)

- 마일스톤(권장):
- 주차 1-2: AVCAffe 접근/다운로드·전처리 파이프라인 고정 15
- 주차 3-4: FP32 baseline 재현 + 학생모델 학습
- 주차 5-6: PTQ/QAT/KD + HEF 생성/컴파일 이슈 해결(unsupported op 제거) 60
- 주차 7-8: Pi 실시간 통합 + 벤치마크 자동화 56
- 주차 9+: 논문(실험/통계/재현 패키지) + 데모 앱(MVP 확장 여지)
- 장비: Raspberry Pi 5 + AI Kit(Hailo-8L) + 카메라/마이크 + 전력계(권장) 61
- 리스크: 패키지 버전 불일치, HEF 컴파일 실패(unsupported layer), 열/스로틀링, 데이터 접근 만료. 대응은 “버전 고정·모델 단순화·쿨링·다운로드 자동화”로 문서화. 62

참고문헌·우선순위 소스

- Raspberry Pi AI Kit(13 TOPS, Hailo-8L, M.2 HAT+ 구성) 24
- Raspberry Pi AI Kit 설치(`hailo-all`, PCIe Gen3 권장, `hailortcli fw-control identify`) 48
- AVCAffe 논문/부록(라벨 구성, baseline 전처리·스케줄) 63
- AVCAffe 공식 저장소(디렉터리 구조, face crops, train/val, no_audio_files, public_face_ids, 접근 절차) 15
- Hailo DFC User Guide(ONNX→HAR, optimize(INT8), compile(HEF), profiler 전력 추정, hw_arch=hailo8l, 지원 레이어/activation) 64
- Hailo 65 HailoRT 리포(구성요소·Hailo-8L은 hailo8 브랜치 사용, pyHailoRT/hailonet/라이선스) 66
- Hailo 소프트웨어 구성요소(GStreamer hailonet 다중 사용/병렬 네트워크) 36
- TAPPAS User Guide(GStreamer pipeline 예시, multi-network 예제) 35
- CREMA-D / RAVDESS 데이터셋 메타 67
- 지식증류/압축/정수 양자화 고전 문헌 68
- WER/CER/RTF 정의 69
- DeLong(AUC 비교), Bland-Altman(방법 비교) 70
- (한국어 데이터) AI-Hub 71 감정 대화 음성 데이터 포맷/메타(48kHz, csv 메타에 ASR 결과) 12
- (데이터 플랫폼) Kaggle 72 RAVDESS/CREMA-D 미러(다운로드 편의) 73
- (선택 ASR) OpenAI 74 Whisper MIT 라이선스 공개 75
- 내부 파일 참조:

1 24 datasheets.raspberrypi.com

<https://datasheets.raspberrypi.com/ai-kit/raspberry-pi-ai-kit-product-brief.pdf>

2 3 6 9 17 18 19 63 <https://arxiv.org/pdf/2205.06887>

<https://arxiv.org/pdf/2205.06887>

4 8 20 22 23 25 26 27 30 31 32 33 34 37 55 58 60 64 https://mmmsk.ai.kr/Projects/Embedded-AI/files/hailo_dataflow_compiler_v3.27.0_user_guide.pdf

https://mmmsk.ai.kr/Projects/Embedded-AI/files/hailo_dataflow_compiler_v3.27.0_user_guide.pdf

5 35 39 52 71 f.hubspotusercontent30.net

<https://f.hubspotusercontent30.net/hubfs/3383687/TAPPAS%20User%20Guide.pdf>

7 36 57 <https://hailo.ai/products/hailo-software/hailo-ai-software-suite/>

<https://hailo.ai/products/hailo-software/hailo-ai-software-suite/>

10 67 72 CREMA-D: Crowd-sourced Emotional Multimodal Actors Dataset

https://pmc.ncbi.nlm.nih.gov/articles/PMC4313618/?utm_source=chatgpt.com

- 11 The Ryerson Audio-Visual Database of Emotional Speech ...
https://journals.plos.org/plosone/article?id=10.1371%2Fjournal.pone.0196391&utm_source=chatgpt.com
- 12 <https://aihub.or.kr/aihubdata/data/view.do?dataSetSn=263>
<https://aihub.or.kr/aihubdata/data/view.do?dataSetSn=263>
- 13 <https://pmc.ncbi.nlm.nih.gov/articles/PMC9823940/>
<https://pmc.ncbi.nlm.nih.gov/articles/PMC9823940/>
- 14 15 41 <https://github.com/pritamqu/AVCAffe>
<https://github.com/pritamqu/AVCAffe>
- 16 50 62 74 <https://www.raspberrypi.com/documentation/computers/ai.html>
<https://www.raspberrypi.com/documentation/computers/ai.html>
- 21 AI software - Raspberry Pi Documentation
https://www.raspberrypi.com/documentation/computers/ai.html?utm_source=chatgpt.com
- 28 68 <https://arxiv.org/abs/1503.02531>
<https://arxiv.org/abs/1503.02531>
- 29 <https://arxiv.org/abs/1510.00149>
<https://arxiv.org/abs/1510.00149>
- 38 <https://pubmed.ncbi.nlm.nih.gov/2868172/>
<https://pubmed.ncbi.nlm.nih.gov/2868172/>
- 40 <https://github.com/wiseman/py-webrtcvad>
<https://github.com/wiseman/py-webrtcvad>
- 42 75 <https://github.com/openai/whisper>
<https://github.com/openai/whisper>
- 43 59 69 <https://www.nist.gov/document/2020opensat20evaluationplanv16>
<https://www.nist.gov/document/2020opensat20evaluationplanv16>
- 44 https://lightning.ai/docs/torchmetrics/stable//text/char_error_rate.html
https://lightning.ai/docs/torchmetrics/stable//text/char_error_rate.html
- 45 <https://learn.microsoft.com/en-us/azure/ai-services/speech-service/embedded-speech-performance-evaluations>
<https://learn.microsoft.com/en-us/azure/ai-services/speech-service/embedded-speech-performance-evaluations>
- 46 <https://www.openslr.org/12>
<https://www.openslr.org/12>
- 47 48 49 56 61 <https://www.raspberrypi.com/news/how-to-set-up-the-raspberry-pi-ai-kit-with-raspberry-pi-5/>
<https://www.raspberrypi.com/news/how-to-set-up-the-raspberry-pi-ai-kit-with-raspberry-pi-5/>
- 51 66 <https://github.com/hailo-ai/hailort>
<https://github.com/hailo-ai/hailort>
- 53 70 <https://pubmed.ncbi.nlm.nih.gov/3203132/>
<https://pubmed.ncbi.nlm.nih.gov/3203132/>
- 54 https://en.wikipedia.org/wiki/McNemar%27s_test
https://en.wikipedia.org/wiki/McNemar%27s_test

65 73 Audio-Visual Database (RAVDess)

https://www.kaggle.com/datasets/orvile/ravdess-dataset?utm_source=chatgpt.com